TA-MPQ: Structured Exact-Budget Mixed Precision for Task-Aware LLM Quantization

Aaron Yu · Letian Zhang · Tim Gao
Carnegie Mellon University · 15-442/15-642: Machine Learning Systems

Motivation & Problem

1

Extreme quantization (uniform INT4 footprint) **may be needed** for local LLM inference under tight memory. At INT4, every bit counts — but uniform allocation wastes bits on insensitive layers and degrades quality on sensitive ones.

- Reallocate precision across groups while staying within the **uniform INT4 footprint**
- Different tasks value different layers — the best policy is workload-aware

Uniform INT4

INT4 | INT4 | INT4 | INT4 | INT4 | INT4 | INT4 | INT4

Mixed (TA-MPQ)

INT8 | INT8 | INT4 | INT4 | INT2 | INT2 | INT2 | INT2

Same total budget · different allocation

Core Idea

2

We profile each linear group with **TAQ-KL-Lite**: inject simulated quantization noise at the group's output and measure KL divergence against the BF16 baseline at INT2/INT4/INT8. For each group we score a *hypothetical INT4→INT8 promotion* by `sensitivity benefit / size increase` (i.e. `benefit / (4 × param_count)` — value per bit of promotion cost). Ranking by this score collapses a **permutation problem** (which order to promote in) into a **combination problem** (which top-*k* of the ranked list to promote), and gives us an **exact-budget policy frontier**:

- **High-sensitivity groups** → promoted to `INT8`
- **Low-value large groups** → demoted to `INT2` to pay for the INT8 cost
- **Remaining groups** → stay at `INT4`

> Every candidate stays within the **uniform INT4 raw weight footprint**. Feasible policies form a **discrete combinatorial space.**

Per-group sensitivity profiler (TAQ-KL-Lite)
Inject simulated quant noise at each group → KL vs BF16 logits

↓

Rank linear components by benchmark sensitivity
Normalize by true component size

↓

Assign precision by sensitivity rank

INT8	INT4	INT2
Top-sensitive	Remaining	Budget offset

↓

Exact budget constraint enforced
Total raw weight footprint = uniform INT4 size

↓

Structured Policy Frontier
Spectrum from max-INT8 (INT2 offsets the budget) → uniform INT4

Why Not Evolutionary Search?

3

We initially looked at **evolutionary search** — bit allocation under a budget is a search in a discrete combinatorial space, the canonical setting for EAs. Two issues broke it:

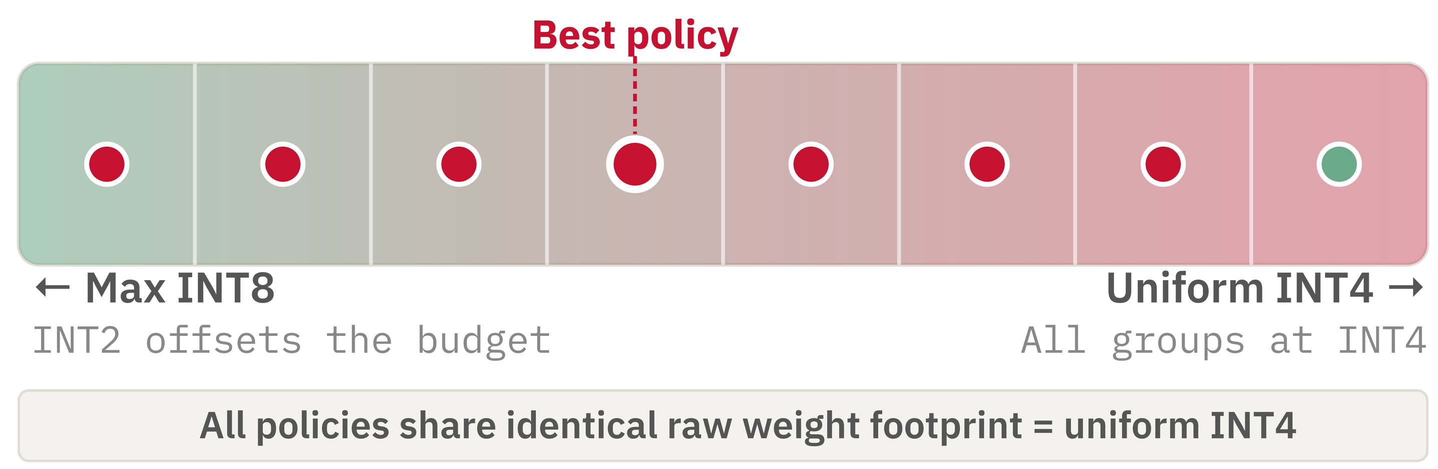
- **Budget violations break inheritance.** Crossover and single-group mutations both routinely produce over-budget children. The repair step rewrites so many assignments that the child no longer inherits parent structure — eliminating EA's core advantage.
- **Cost per evaluation is high.** Each candidate is a real PTQ artifact + benchmark run; EA-scale populations aren't affordable here.

A ranking-based greedy frontier sidesteps both — collapsing the combinatorial space into a 1D coarse-to-fine sweep with predictable cost.

Policy Frontier

4a

Each point on the frontier is an **exact-budget policy** at the uniform INT4 footprint. As we promote more groups to INT8, we must demote others to INT2 to stay within the INT4 budget — spanning a full spectrum. Adjacent policies tend to have similar benchmark performance, so the space is discrete but behaves like a **nearly continuous function** — justifying a coarse-to-fine search where a good sector reliably contains a good policy.



Structured Exact-Budget Coarse-to-Fine Search

4b

Round 1 — Coarse

S1 · S2 · S3★ · S4 · S5 · S6 · S7 · S8

- 8 evenly spaced frontier sectors
- 1 representative policy per sector
- Select the best sector
- *Ties → prefer higher INT4 fraction*

Round 2 — Fine

P1 · P2★ · P3 · P4

- 4 evenly spaced policies inside winning sector
- Evaluate all 4
- Select the best policy

*Total search cost: **12 PTQ evals** (8 coarse + 4 fine; each eval is a real quantized model)*

Experimental Setup

5a

MODEL
`Qwen3.5-9B`

BIT SPACE
`INT2` `INT4` `INT8`

ALLOCATION GRANULARITY
One bit-width per **linear component** within each transformer block
e.g. q/k/v/o, gate/up/down — ~290 quantizable components for Qwen3.5-9B

BUDGET
Raw weight footprint = uniform INT4

BASELINES
BF16 — unquantized reference
Uniform INT4 — every linear group at INT4 via `llmcompressor.oneshot` (RTN, W4A16, group-size 128, symmetric)
Mixed (TA-MPQ) is built with the same quantization backend, so INT4-vs-Mixed comparisons isolate the allocation policy.

> **Why no GPTQ / AWQ?** Layering weight-refinement on top of RTN would perturb the per-group sensitivity signal that drives allocation, confounding the variable we want to measure. We isolate *bit allocation* here and leave the combination for future work.

PROMPTING · PER BENCHMARK
MMLU-coding: `simple_evals` CoT
HumanEval / +: `official_evalplus` (no CoT)
BigCodeBench-H: `official_bigcodebench` (no CoT)
MATH-500: `simple_evals_nonthinking`
`All runs use greedy decoding`

EVAL SPLIT
Sensitivity calibration: 64 prompts from `first 300` (MMLU-coding) / `q301–400` (MATH-500)
Coarse search: 25 Qs from `first 300`
Fine search: 50 Qs from `first 300` (disjoint from coarse)
Final eval: held-out `last 100`
Held-out final eval prevents look-ahead bias from sensitivity profiling and search

SEARCH → EVALUATION

SEARCHED ON
MATH-500 `max=4096`
MATH-500 `max=20000`
MMLU-coding
Produces one math policy + one code policy

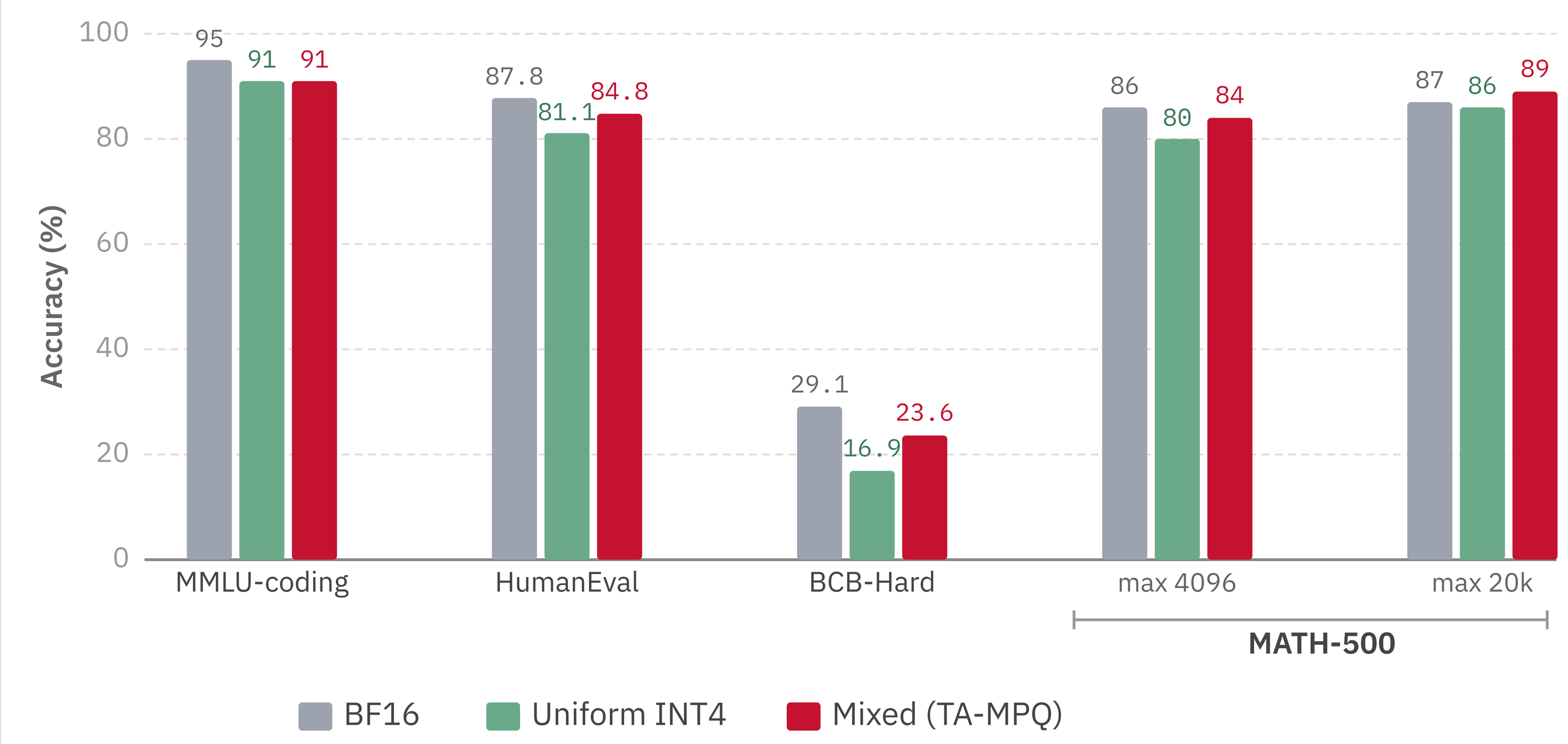
EVALUATED ON
Math policy → MATH-500 @4096, @20000
Code policy → MMLU-coding, HumanEval, HumanEval+, BigCodeBench-Hard

> **Cross-benchmark transfer:** the code policy was searched *only* on MMLU-coding, then evaluated on HumanEval, HumanEval+, and BigCodeBench-Hard — testing whether coding skill preserved by one search transfers across coding benchmarks.

Results & Interpretation

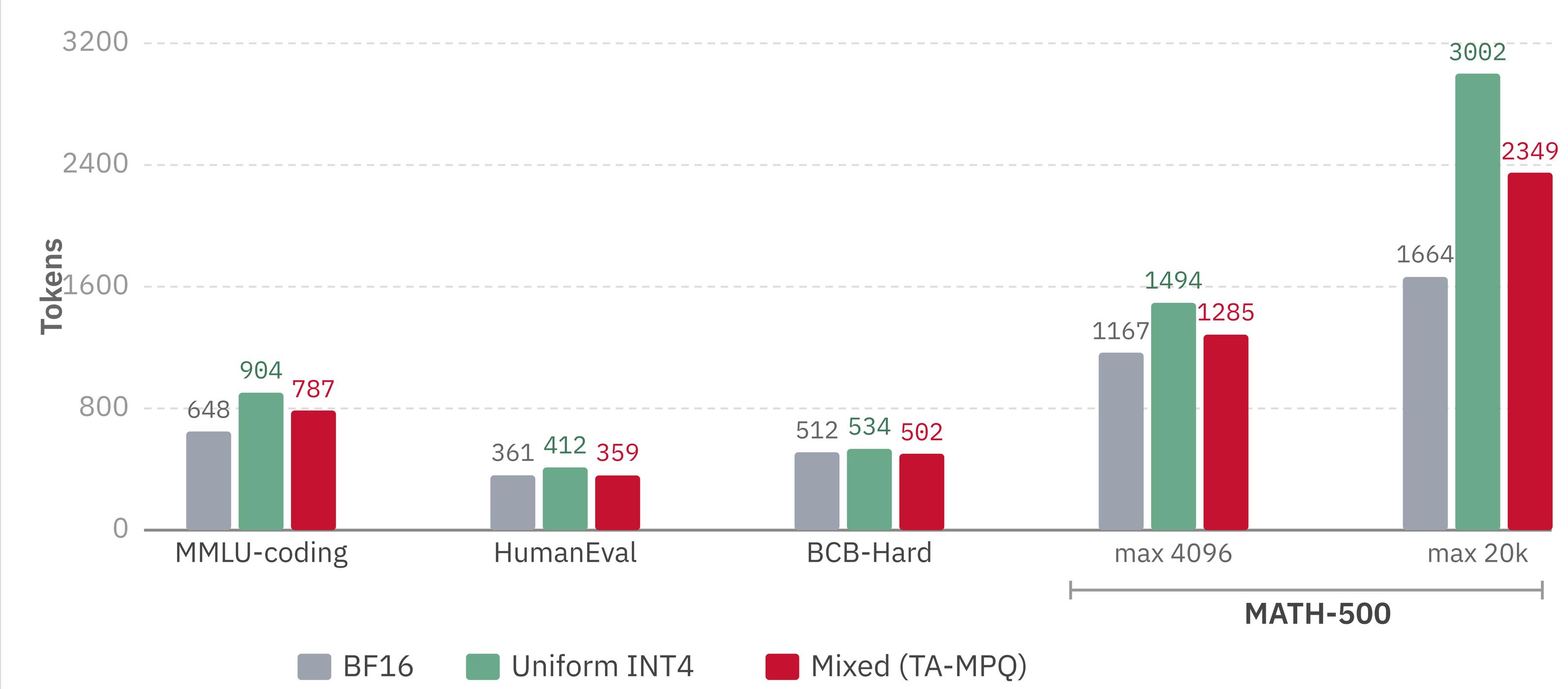
5b

ACCURACY BY BENCHMARKS WE TESTED



> **Observation.** Mixed precision matches or beats uniform INT4 on every benchmark.

AVG TOKEN USAGE BY BENCHMARKS WE TESTED



> **Observation.** Mixed uses fewer tokens than uniform INT4 on every benchmark. **Conjecture:** higher precision at sensitive layers keeps reasoning on-track — pending verification.

Limitations & Future Work

6

CURRENT LIMITATIONS

- Benchmark sizes are modest; small differences should be interpreted carefully
- We use `llmcompressor.oneshot` rather than an optimized INT4 serving stack — so **token usage and accuracy** are more trustworthy comparison points than raw latency

POTENTIAL FUTURE WORK

- Combine TA-MPQ allocation with **GPTQ / AWQ** weight refinement — test whether allocation gains stack with sensitivity tuning
- Larger model families and longer experiments
- Layer-level vs. component-level granularity comparison
- Optimize directly for token usage, not only accuracy
- Controlled cross-task experiments to test true task specificity